

Avellaneda–Stoikov Market Making Engine

C++ Project

Charbonnier Thibault, Chardigny Lola, Letaconnoux Lauryn

December 23, 2025

Abstract

We use Modern C++ to implement the market making strategy described in the Avellaneda–Stoikov model: *"High-frequency trading in a limit order book"*, Marco Avellaneda & Sasha Stoikov, 2006.

We use an object-oriented architecture with proper use of **abstract classes** and **polymorphism**. We also use safe memory management via **smart pointers** and **move semantics**.

We construct a market making engine which

- simulates a **stream of market ticks** (market prices), either from:
 - a synthetic stochastic model by generating a Brownian Motion,
 - a CSV to “replay” a real trading day,
 - and is designed to be extendable with other sources of data;
- computes **mid/bid/ask prices and spread** of a market-making strategy such as:
 - the Avellaneda–Stoikov model, based on an indifference price of the market-maker,
 - the benchmark strategy, which considers the market price as the mid price,
 - and is designed to be extendable with other strategies;
- models **market order arrivals** via the **inventory evolution of the market maker** as a Poisson process. We modeled two inventories: sold stocks and bought stocks. The Poisson distribution intensities increase as quotes move closer to the mid price;
- simulates a full **market-making session** (e.g. one trading day at high frequency), tracks inventory and P&L, and compares;
- simulates **Monte Carlo simulations** of market-making sessions with different sets of market-maker parameters (e.g. risk aversion), and computes mean P&L, mean standard deviation of each set.

Contents

1	Model in Theory	3
1.1	Price process	3
1.2	Avellaneda–Stoikov inventory-aware quoting	3
1.3	Order simulation: Poisson arrivals	3
2	Model in C++	4
2.1	Core	4
2.2	Strategy	4
2.3	Market Data	4
2.4	Inventory	5
2.5	Simulation	5

1 Model in Theory

1.1 Price process

The market price S_t of the traded asset is modeled as a simple stochastic process, e.g.:

- **Additive Brownian motion** (discretized):

$$S_{t+\Delta t} = S_t + \mu\Delta t + \sigma\sqrt{\Delta t} Z_t, \quad Z_t \sim \mathcal{N}(0, 1)$$

or alternatively with a geometric Brownian motion.

Drift parameter μ and volatility parameter σ can be chosen.

1.2 Avellaneda–Stoikov inventory-aware quoting

The core idea of the Avellaneda–Stoikov model is to fix an indifference price of the market maker between sell, buy and do nothing, and then compute a spread centered around this price based on its **risk-aversion**.

Let:

- S_t be the current market price,
- q_t be the current inventory,
- γ be the risk-aversion coefficient,
- σ be the volatility,
- T the terminal horizon (end of the trading session).

A closed-form approximation for the **reservation price** / **indifference price** is:

$$r_t = S_t - q_t \gamma \sigma^2 (T - t)$$

this is the **mid price** of the strategy.

A closed-form approximation for the **optimal half-spread** is:

$$\delta_t = \frac{\gamma \sigma^2 (T - t)}{2} + \frac{1}{\gamma} \log \left(1 + \frac{\gamma}{k} \right)$$

where k is the order arrival intensity parameter.

Then **bid and ask prices** are:

$$\text{bid}_t = r_t - \delta_t$$

$$\text{ask}_t = r_t + \delta_t$$

1.3 Order simulation: Poisson arrivals

Order arrivals on the market-maker's quotes are modeled via the evolution of the inventory as **independent Poisson processes** on the bid and ask sides.

- q_t^{ask} be the current inventory of stocks sold,
- q_t^{bid} be the current inventory of stocks bought,

The **order arrival intensity** is:

$$\lambda_t^{\text{bid}}(\delta_t) = A \exp(-k\delta_t) = \lambda_t^{\text{ask}}(\delta_t)$$

where:

- A intensity parameter,
- k intensity parameter of arrival of orders,
- δ is the current half-spread.

$$q_t^{\text{ask}} \sim \text{Poisson}(\lambda_t^{\text{ask}}(\delta))$$

$$q_t^{\text{bid}} \sim \text{Poisson}(\lambda_t^{\text{bid}}(\delta))$$

The current inventory is defined as:

$$q_t = q_t^{\text{bid}} - q_t^{\text{ask}}$$

2 Model in C++

2.1 Core

- **Tick**: Structure representing a market data tick (time, price).
- **Quote**: Structure representing a market-maker quote on a trading instrument (mid, bid, ask, delta).
- **BookState**: Structure representing the state of a market-maker book at a given time (time, stocks sold, stocks bought, inventory, cash).
- **BookSnapshot**: Structure representing a snapshot of the market-maker's situation at a specific time (tick, state, quote).
- **TradingHistory**: Structure representing the history of the market-maker's books and quotes over a trading period.

2.2 Strategy

- **IStrategy**: Abstract class of strategies. Interface for market-making strategy implementations.
 - **AsStrategy**: Inherits from **IStrategy**. Class for the Avellaneda-Stoikov strategy.
 - **Benchmark**: Inherits from **IStrategy**. Class for the Benchmark strategy, which considers market price as the mid price.

2.3 Market Data

- **IMarketDataStream**: Abstract base class for data streams. Interface for different types of market data sources.
 - **BrownianStream**: Inherits from **IMarketDataStream**. Generates market data following a Geometric Brownian Motion using the Black-Scholes model.
 - **CSVStream**: Inherits from **IMarketDataStream**. Market data stream that reads historical price data from a CSV file.

2.4 Inventory

- **IInventoryModel**: Abstract class. Interface for inventory models used in market making simulations.
 - **PoissonInventory**: Inherits from **IInventoryModel**. Simulation of the inventory movement as a Poisson process.

2.5 Simulation

- **MarketSimulator**: Class that simulates market-making activity over a stream of market data.
 - Uses pointer to **IStrategy** for applying polymorphism (e.g. `generate_quote()`, `get_current_book()`).
 - Uses pointer to **IMarketDataStream** for applying polymorphism (e.g. `next_tick()`, `has_next()`).
 - Uses pointer to **IInventoryModel** for applying polymorphism (e.g. `update_inventory()`).
- **MCPParameterSet**: Structure of parameter set for configuring market simulations via a single Monte Carlo simulation.
- **SimulationResult**: Structure of results from a single simulation (e.g. final P&L, final cash...).
- **MCStats**: Structure of aggregate statistics computed from many Monte Carlo simulation results (e.g. mean P&L,...).
- **MCEngine**: Class computing the Monte Carlo engine for running multiple market simulations with different parameters.
 - Uses pointer to **MCPParameterSet**.
 - Uses pointer to **SimulationResult**.
 - Uses pointer to **MarketSimulator**.
- **Visualizer**: Class for visualizing and exporting trading simulation results.